

Übungsblatt 8: Multicore-Synchronisation und Safety

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

In dieser Laborübung verwandeln Sie Ihren Code in eine ausfallsichere Serien-Architektur. Sie ersetzen das naive „Ping-Pong“-Protokoll aus Termin 6 durch echte Hardware-Semaphoren (**HSEM**), um asynchrone Multicore-Zugriffe auf der Ebene der Bus-Matrix physikalisch abzusichern. Im zweiten Teil implementieren Sie Safety-Mechanismen: Sie überwachen das Timing der FreeRTOS-Task mit einem **Window Watchdog (WWDG)** und programmieren einen Safe Shutdown über den **Early Wakeup Interrupt (EWI)**.

Aufgabe 1: Asynchrone Multicore-Synchronisation (HSEM)

Wir entfernen die alten Handshake-Flags (`m7_ready`, `m4_ready`) und lassen beide Kerne asynchron und so schnell wie möglich denselben Zähler im Shared Memory erhöhen. Nutzen Sie weiterhin die Shared-Variable `sd` aus `shared_data.h` (Feld `payload_value`, analog zu Übungsblatt 6).

1. Öffnen Sie Ihr Dual-Core-Projekt aus Woche 6 im CubeMX Configurator.
2. Aktivieren Sie unter *System Core* das Modul **HSEM** und weisen Sie es **beiden** Kernen (Cortex-M7 und M4) zu. Generieren Sie den Code.
3. Entfernen Sie in beiden `main.c` / `freertos.c`-Dateien die alten `while (sd.m7_ready == 0)`- bzw. `while (sd.m4_ready == 0)`-Schleifen.
4. Implementieren Sie auf beiden Kernen den Zugriff auf `sd.payload_value++`, abgesichert durch `HAL_HSEM_FastTake(0)` und `HAL_HSEM_Release(0, 0)`.
5. Denken Sie daran: Der M4 darf im FreeRTOS-Kontext bei einem belegten Schloss **nicht** durch eine leere `while`-Schleife (Spinlock) blockieren. Fügen Sie hier ein `osDelay(1)` ein. (*Labor-Kompromiss: In der Vorlesung wurde der elegantere HSEM-Interrupt-Pfad vorgestellt. Hier reicht die pragmatische Stufe mit `osDelay(1)`.*)

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Den Window Watchdog (WWDG) integrieren

Wir härten das System nun gegen Software-Hänger und fehlerhafte (zu schnelle) Ausführungen ab.

1. Öffnen Sie CubeMX und aktivieren Sie den **WWDG** für den Cortex-M4.
2. Konfigurieren Sie den Watchdog:
 - *WWDG clock prescaler*: 8
 - *Window value*: 80 (das erlaubte Fenster beginnt erst bei Zählerstand 80)
 - *Free-running downcounter value*: 120 (Startwert)
3. Aktivieren Sie im Reiter *NVIC Settings* zwingend den **Early Wakeup Interrupt (EWI)**. Generieren Sie den Code.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Watchdog füttern und Crash-Tests

Kalibrieren Sie das Timing Ihrer Task und testen Sie das Sicherheitsnetz.

1. Fügen Sie in die Endlosschleife Ihrer M4-Task ein `osDelay(15)` ein. Rufen Sie danach `HAL_WWDG_Refresh(&hwwdg)` auf. Das System sollte nun stabil laufen.
2. **Crash-Test 1 (Late Phase)**: Ändern Sie das Delay auf `osDelay(50)`. Flashen Sie den Code und erklären Sie, was passiert.
3. **Crash-Test 2 (Early Phase)**: Ändern Sie das Delay auf `osDelay(5)`. Flashen Sie den Code und erklären Sie, was nun passiert und warum dieses Verhalten für die Safety wichtig ist.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Safe Shutdown via Early Wakeup Interrupt (EWI)

Bevor der WWDG das System beim Absturz hart neu startet, wollen wir das System in einen sicheren Zustand (*Safe State*) überführen.

1. Überschreiben Sie in der `main.c` des Cortex-M4 die Funktion `HAL_WWDG_EarlyWakeupCallback(WWDG_Handle_t *hwwdg)`.
2. Schalten Sie innerhalb dieses Callbacks die **rote** Fehler-LED des Boards ein (Nucleo-H755ZI-Q: **LD3 an PB14**, nicht die gelbe LD2 an PE1).
3. Führen Sie Crash-Test 1 (Late Phase) erneut aus. Was beobachten Sie an der roten LED? Darf der Watchdog in diesem Callback noch gefüttert werden?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)